



University of Asia Pacific
Department of Computer Science and Engineering
CSE 450: VLSI Lab

LAB REPORT

Experiment Number: 05

Experiment Title: Design and Simulation of an 8-bit Arithmetic Logic Unit (ALU) for Addition and Subtraction Using NAND-Based Structural VHDL in Xilinx ISE

Submitted by:

Name: Rifah Tasfia

Student ID: 22201126

Section: C1

Submitted to:

Md. Rasheduzzaman

Lecturer

Department of Computer Science and Engineering

Date of Submission: 01/10/2026

1. Experiment Name

Design and Simulation of an 8-bit Arithmetic Logic Unit (ALU) for Addition and Subtraction Using NAND-Based Structural VHDL in Xilinx ISE.

2. Objective

In this experiment, we want to:

1. Understand how basic digital logic gates work.
2. Implement a NAND gate using VHDL dataflow modeling.
3. Build NOT, AND, OR, and XOR gates using only NAND gate instances.
4. Design a Half Adder using XOR and AND gates.
5. Build a Full Adder from two Half Adders and an OR gate, and then extend it to an 8-bit Full Adder.
6. Design a Full Subtractor using two XOR gates, two AND gates, two NOT gates, and one OR gate, and extend it to an 8-bit Subtractor.
7. Design an 8-bit multiplexer (MUX) that selects between two 8-bit data paths.
8. Combine the blocks to make an 8-bit ALU that performs addition and subtraction.
9. Learn structural VHDL and hierarchical digital circuit design.
10. Use Xilinx ISE (ISim) to simulate the circuits and compare the outputs with expected truth-table and calculation results.

3. Apparatus / Hardware & Software Requirements

- List all required tools and components:

- Xilinx ISE 14.7
- VHDL
- ISim Simulator
- A computer for design and simulation

4. Introduction

An Arithmetic Logic Unit (ALU) is a major part of a processor. It works with binary data and performs arithmetic and logic operations. In this experiment, an 8-bit ALU was designed to perform two operations: addition and subtraction. A select input is used to choose which operation is active.

The circuit was developed in a hierarchical structural way. First, a NAND gate was created with VHDL dataflow modeling. Because NAND is a universal gate, it was then used to create the NOT, AND, OR, and XOR gates. These gates were used to make a Half Adder, and two Half Adders plus an OR gate were used to make a Full Adder. Eight Full Adders were connected to create the 8-bit Full Adder.

For subtraction, a Full Subtractor was made from two XOR gates, two AND gates, two NOT gates, and one OR gate. Eight Full Subtractors were connected to create the 8-bit Subtractor. Finally, an 8-bit MUX selects either the adder result or the subtractor result. These blocks together form the ALU.

The design hierarchy is:

NAND Gate → Basic Gates (NOT, AND, OR, XOR) → Half Adder → Full Adder / Full Subtractor → 8-bit Adder / 8-bit Subtractor → 8-bit MUX → 8-bit ALU

5. NAND Gate Design

A NAND gate is a universal logic gate. A two-input NAND gate gives the opposite result of an AND operation.

The Boolean expression is:

$$Y = \overline{A \cdot B}$$

Truth Table:

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

The NAND gate was implemented using VHDL Dataflow modeling.

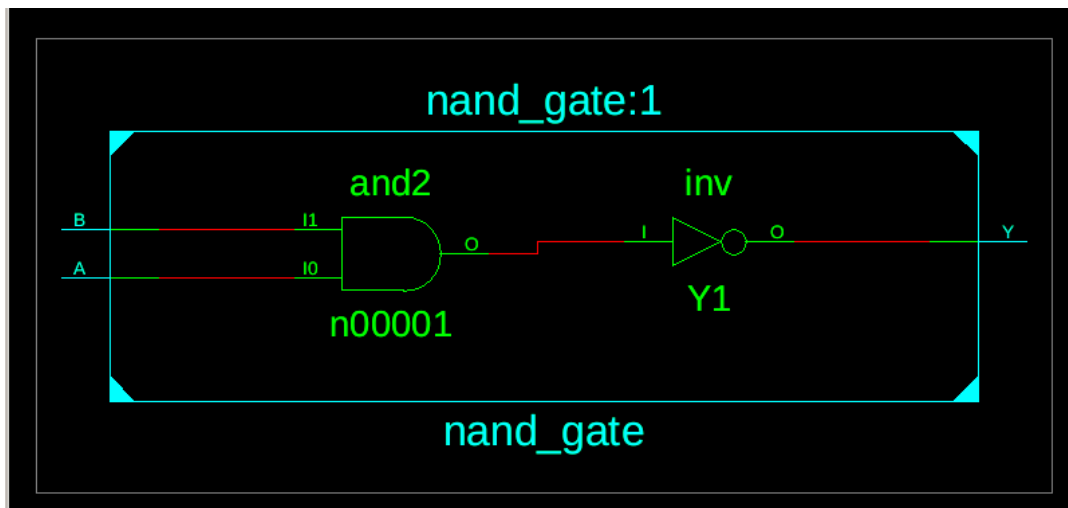


Figure 1: RTL schematic of the NAND gate.

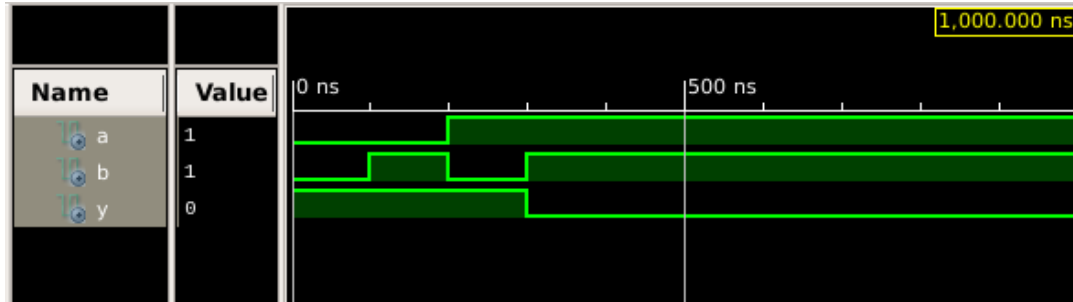


Figure 2: Simulation waveform of the NAND gate.

6. Construction of Basic Gates Using NAND Gates

Since NAND is universal, the other basic gates can be made by connecting NAND gates together. The gates used here are NOT, AND, OR, and XOR. They were implemented with structural VHDL by creating instances of the NAND gate component.

6.1 NOT Gate Using NAND

To make a NOT gate, both inputs of one NAND gate are connected to the same signal. Therefore, $Y = A \text{ NAND } A = \neg A$.

A	Y
0	1
1	0

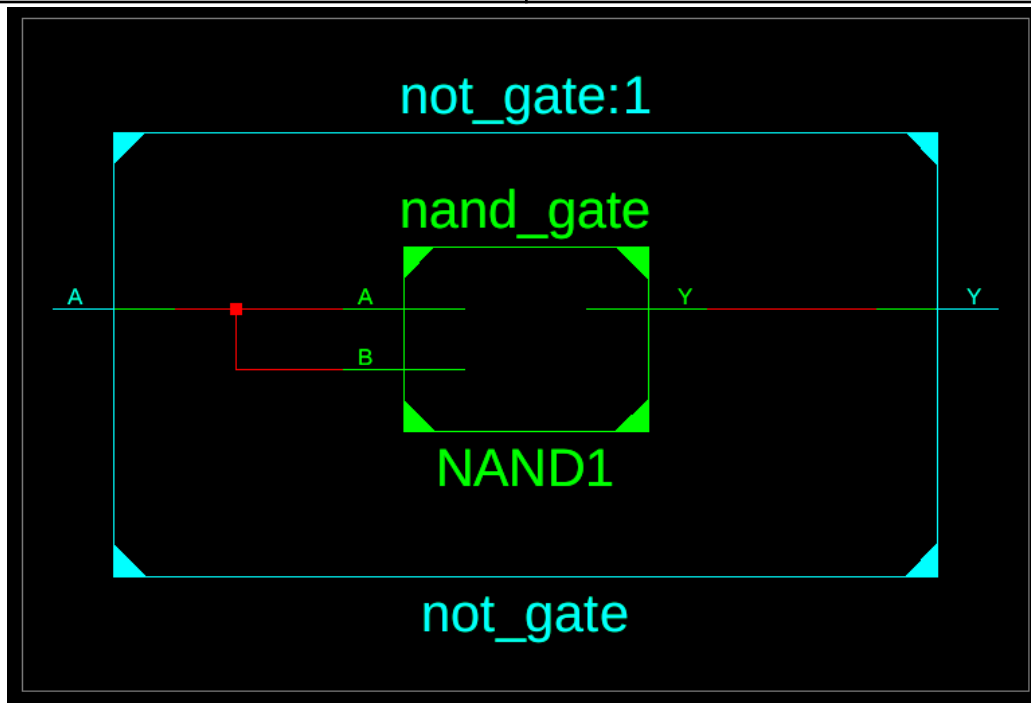


Figure 3: RTL schematic of the NAND-based NOT gate.

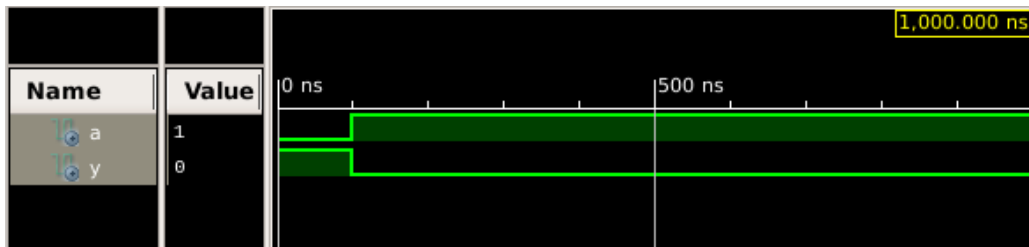


Figure 4: Simulation waveform of the NOT gate.

6.2 AND Gate Using NAND

An AND gate can be made with two NAND gates. First, $N1 = A \text{ NAND } B$. Then, $Y = N1 \text{ NAND } N1$. This produces $Y = A \cdot B$.

NAND gates required: 2

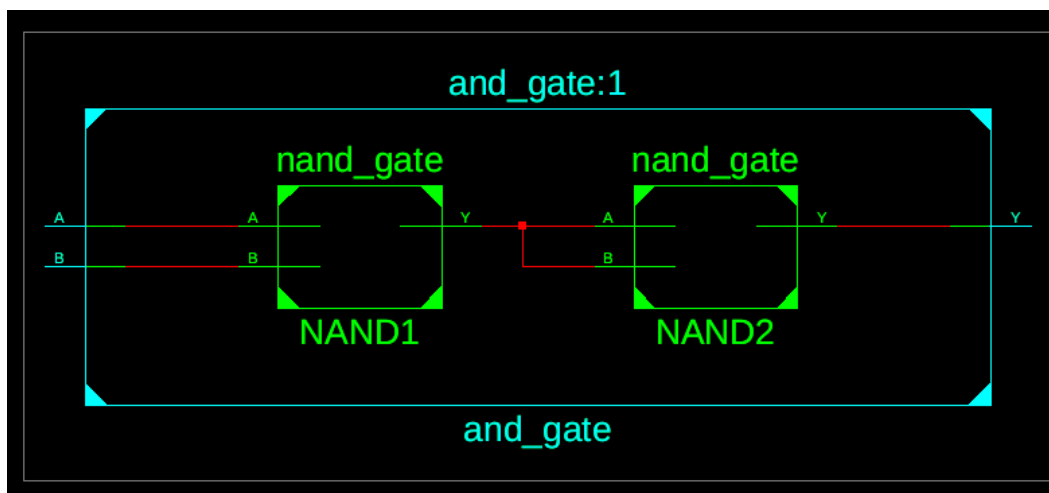


Figure 5: RTL schematic of the NAND-based AND gate.

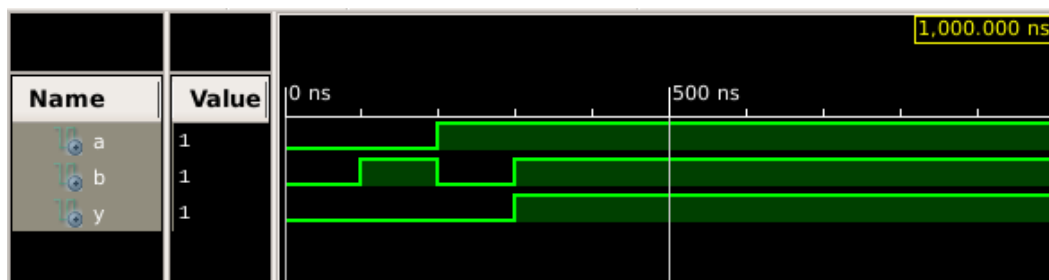


Figure 6: Simulation waveform of the AND gate.

6.3 OR Gate Using NAND Gates

Using De Morgan's theorem, an OR function can be created with three NAND gates. First make the inverted inputs: $N1 = A \text{ NAND } A$ and $N2 = B \text{ NAND } B$. Then $Y = N1 \text{ NAND } N2$.

NAND gates required: 3

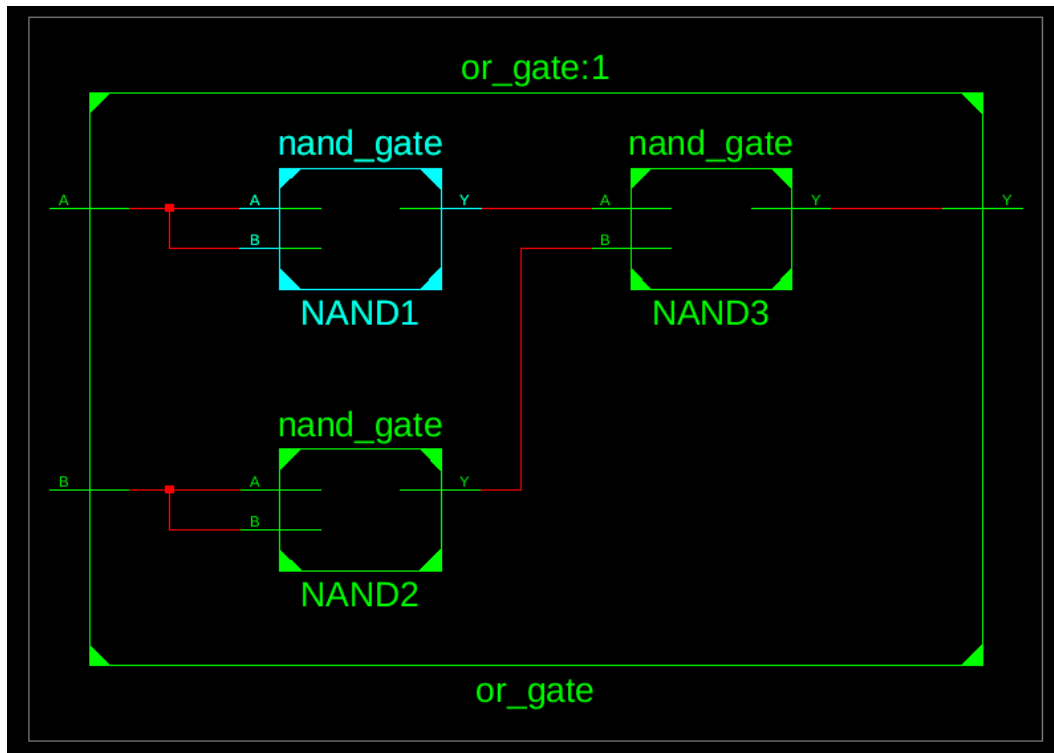


Figure 7: RTL schematic of the NAND-based OR gate.

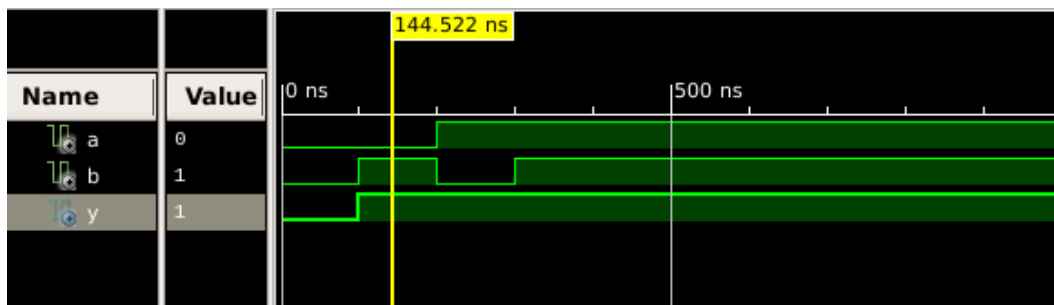


Figure 8: Simulation waveform of the OR gate

6.4 XOR Gate Using NAND Gates

An XOR gate gives logic 1 when its two inputs are different.

The NAND-only design uses four NAND gates:

$$N1 = A \text{ NAND } B$$

$$N2 = A \text{ NAND } N1$$

$$N3 = B \text{ NAND } N1$$

$$Y = N2 \text{ NAND } N3$$

Therefore: $Y = A \oplus B$

A	B	XOR
0	0	0
0	1	1
1	0	1
1	1	0

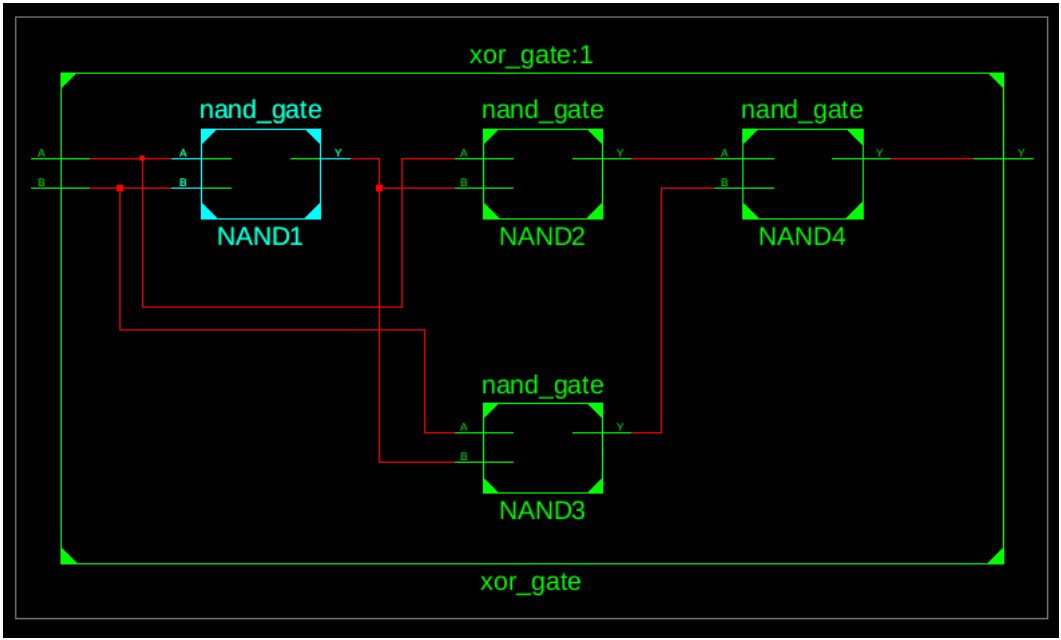


Figure 9: RTL schematic of the NAND-based XOR gate.

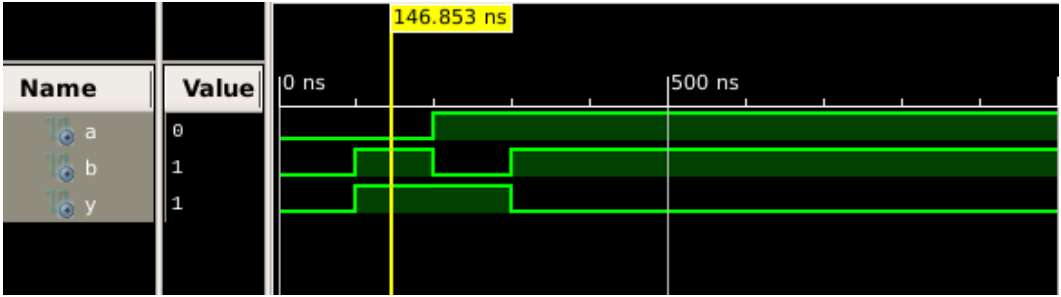


Figure 10: Simulation waveform of the XOR gate.

7. Half Adder Design

A Half Adder is a combinational circuit that adds two one-bit binary numbers.

It has two inputs:

- A
- B

It has two outputs:

- Sum
- Carry

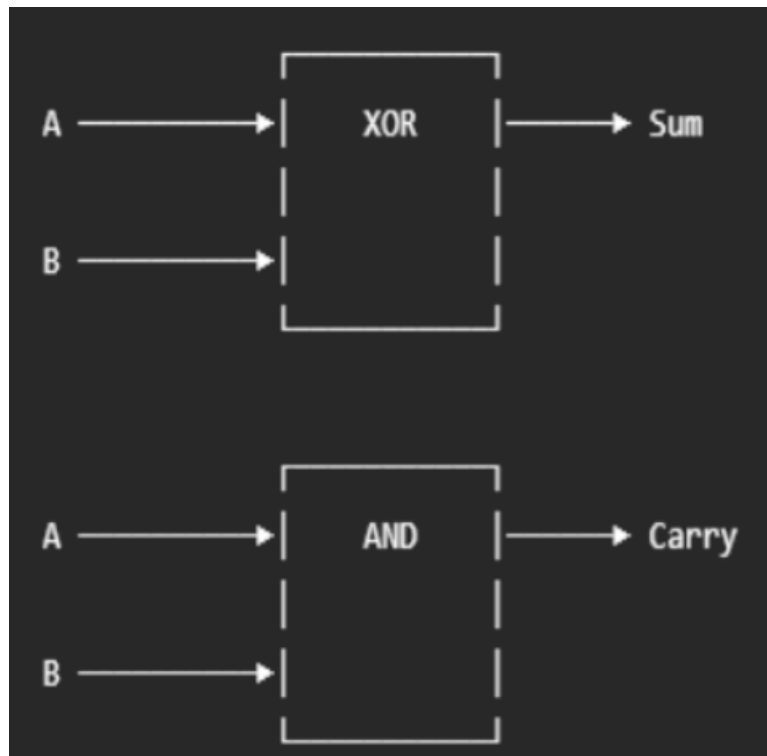
The Boolean expressions are:

$$\text{Sum} = A \oplus B$$

$$\text{Carry} = A \cdot B$$

The Half Adder was designed using the XOR and AND components created earlier.

Block Diagram



Half Adder Truth Table

A	B	Sum	Carry
---	---	-----	-------

0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

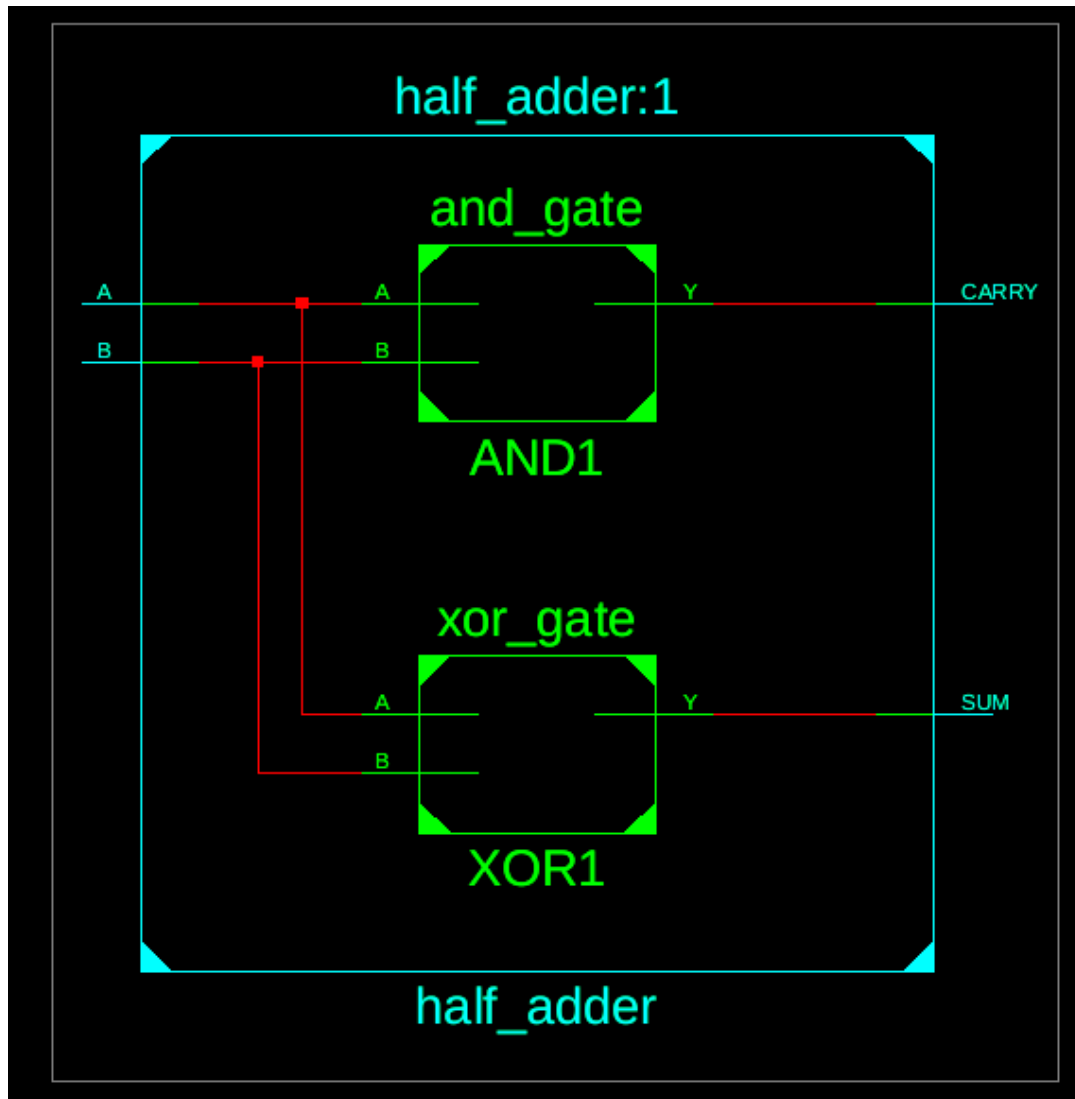


Figure 11: RTL schematic of the Half Adder.

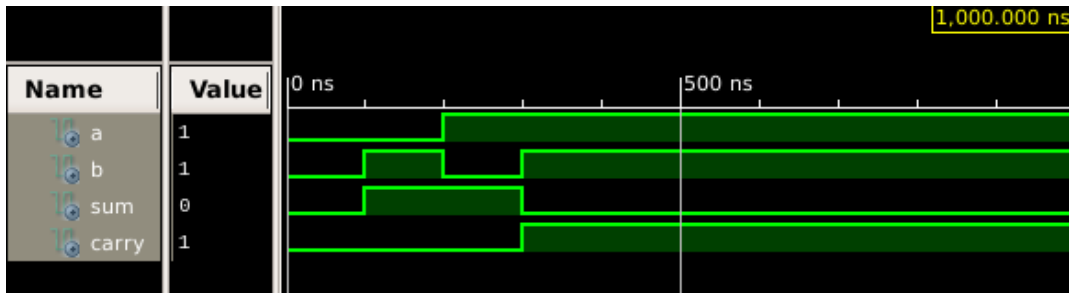


Figure 12: Simulation waveform of the Half Adder.

8. Full Adder Design

A Full Adder adds three one-bit binary values.

Inputs:

- A
- B
- Cin

Outputs:

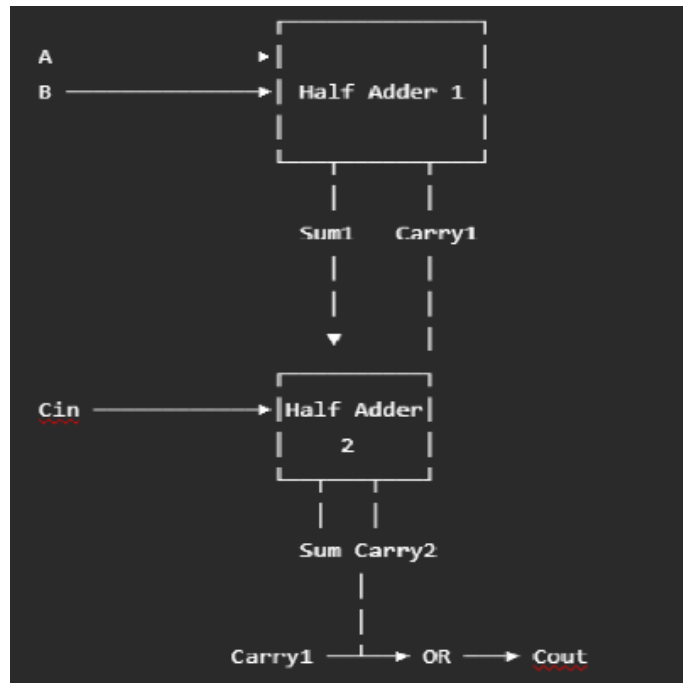
- Sum
- Cout

A Full Adder can be built using:

- Two Half Adders
- One OR gate

The first Half Adder adds A and B. The second Half Adder adds the first Half Adder's Sum with Cin. Finally, the two carry outputs are combined using an OR gate.

8.1 Full Adder Block Diagram



The Full Adder is arranged using two Half Adders and one OR gate.

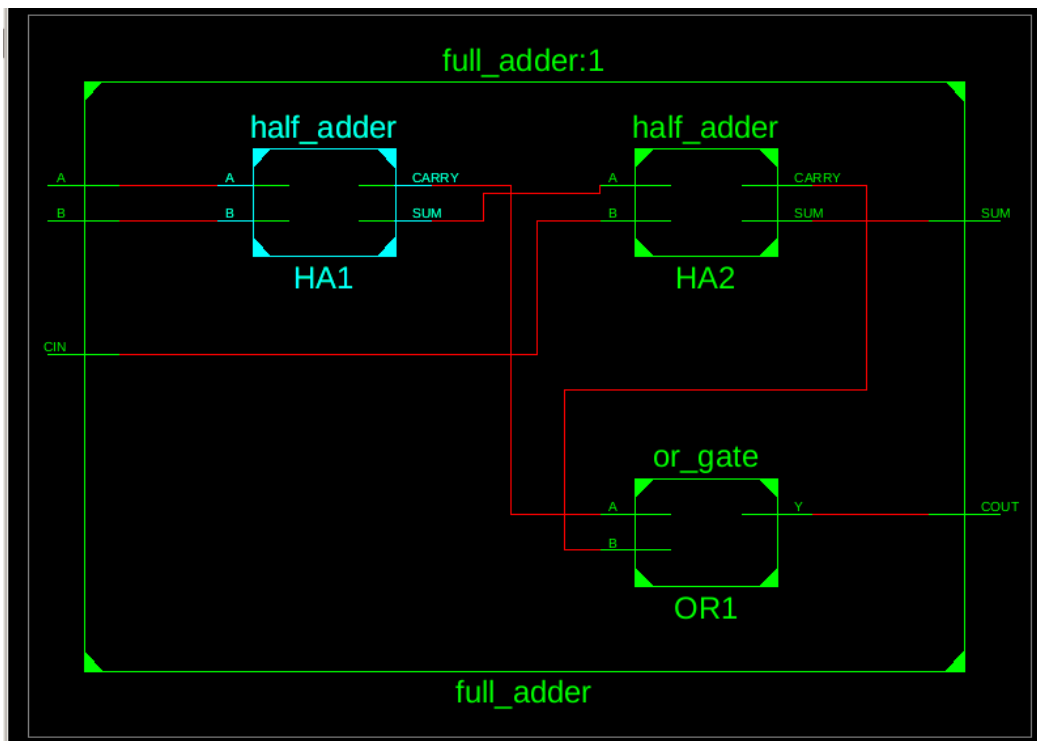


Figure 13: RTL schematic of the complete Full Adder.

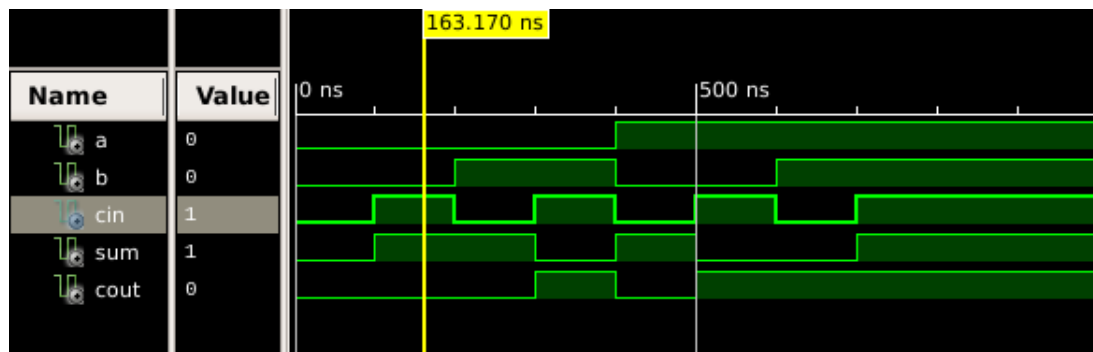


Figure 14: Simulation waveform covering all Full Adder input combinations.

9. Full Adder Logic

The first Half Adder produces:

$$\text{Sum1} = A \oplus B$$

$$\text{Carry1} = A \cdot B$$

The second Half Adder produces:

$$\text{Sum} = \text{Sum1} \oplus \text{Cin}$$

$$\text{Carry2} = \text{Sum1} \cdot \text{Cin}$$

Finally:

$$\text{Cout} = \text{Carry1} + \text{Carry2}$$

Therefore:

$$\text{Sum} = A \oplus B \oplus \text{Cin}$$

$$\text{Cout} = AB + (A \oplus B)\text{Cin}$$

10. 8-bit Full Adder Design

An 8-bit Full Adder adds two 8-bit binary numbers. It is made by connecting eight 1-bit Full Adders in a ripple-carry chain. The carry output of one stage becomes the carry input of the next stage.

- A[7:0] – first 8-bit input
- B[7:0] – second 8-bit input
- Cin – starting carry input
- Sum[7:0] – 8-bit result
- Cout – final carry output, used when the result is greater than 255

For each bit i (0 to 7): $\text{Sum}_i = A_i \oplus B_i \oplus C_i$

$$C_{i+1} = A_i B_i + (A_i \oplus B_i) C_i, \text{ where } C_0 = C_{in} \text{ and } C_{out} = C_8.$$

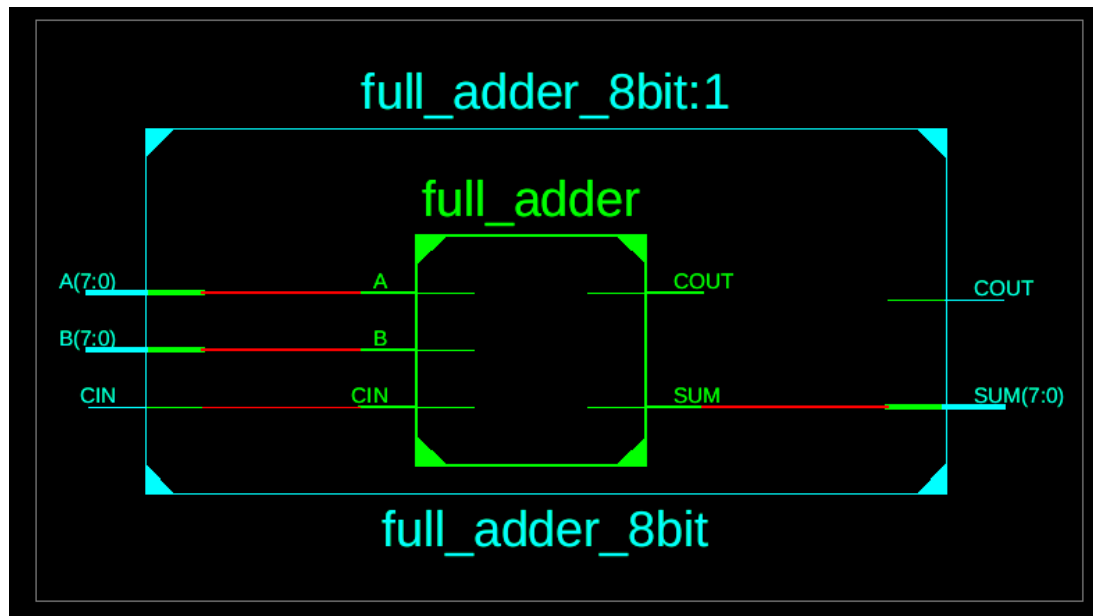


Figure 15: RTL schematic of the 8-bit Full Adder.

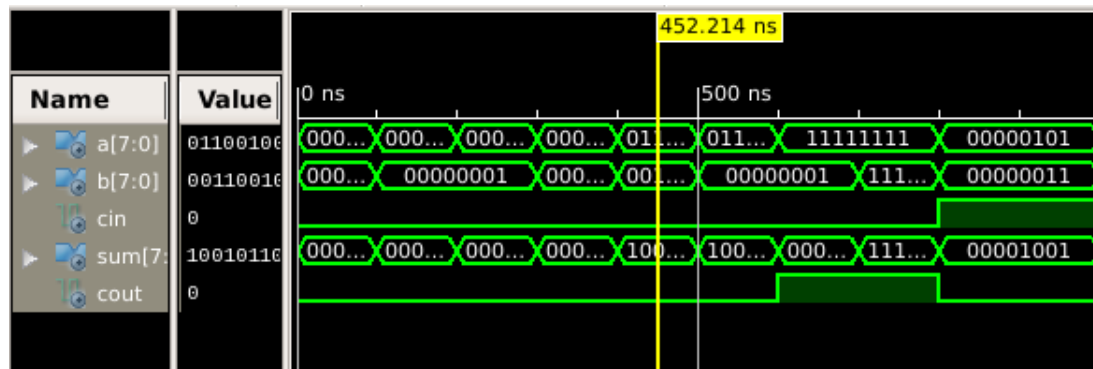


Figure 16: Simulation waveform of the 8-bit Full Adder.

The simulation results should match this truth table for all eight possible input combinations.

11. Full Subtractor Design

A Full Subtractor subtracts B and the incoming borrow Bin from A. It produces a Difference and an outgoing borrow Bout.

- Inputs: A, B, Bin
- Outputs: Diff, Bout

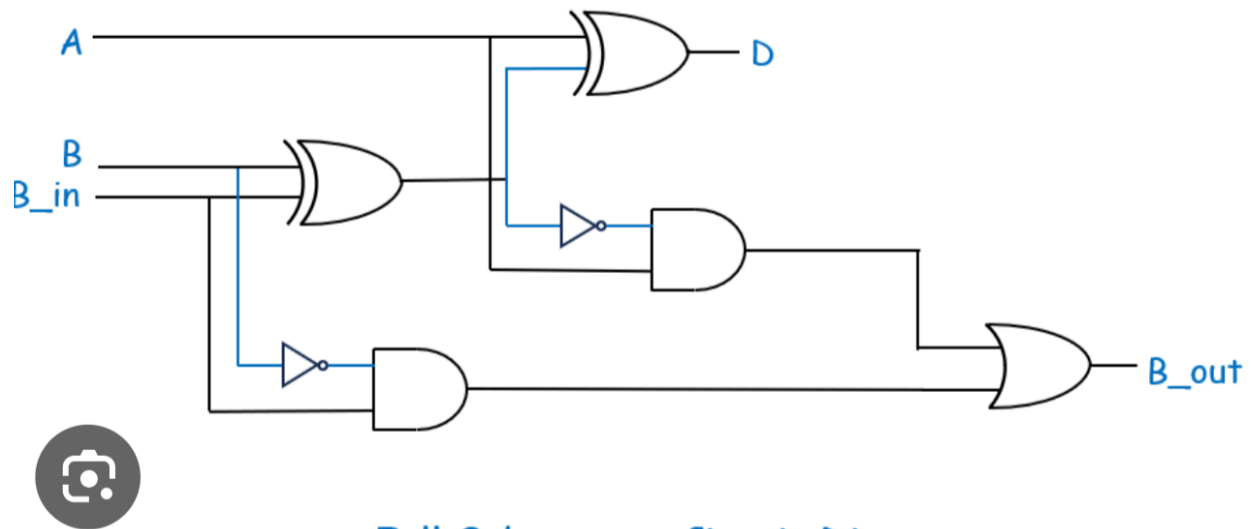
Equations: $\text{Diff} = A \oplus B \oplus \text{Bin}$

$$\text{Bout} = (\neg A \cdot B) + (\neg(A \oplus B) \cdot \text{Bin})$$

Gate	Quantity	Purpose
XOR	2	Create $A \oplus B$ and the Difference
AND	2	Create the two borrow terms
NOT	2	Create $\neg A$ and $\neg(A \oplus B)$
OR	1	Combine the borrow terms into Bout

Internal signals are:

- $X1 = A \text{ XOR } B$
- $\text{Diff} = X1 \text{ XOR } \text{Bin}$
- $N1 = \text{NOT } A$
- $N2 = \text{NOT } X1$
- $T1 = N1 \text{ AND } B$
- $T2 = N2 \text{ AND } \text{Bin}$
- $\text{Bout} = T1 \text{ OR } T2$



Full Subtractor Circuit Diagram

Figure 17: Redrawn gate-level block diagram of the Full Subtractor.

A	B	Bin	Diff	Bout
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

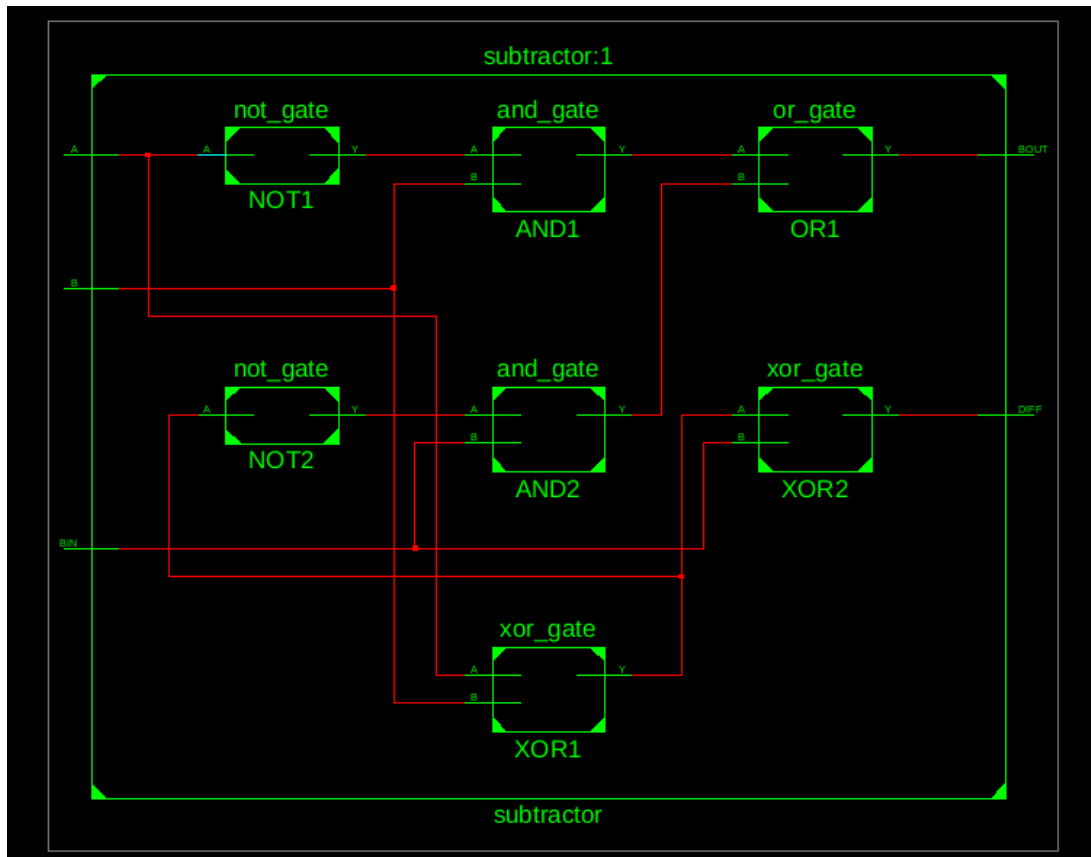


Figure 18: RTL schematic of the Full Subtractor.

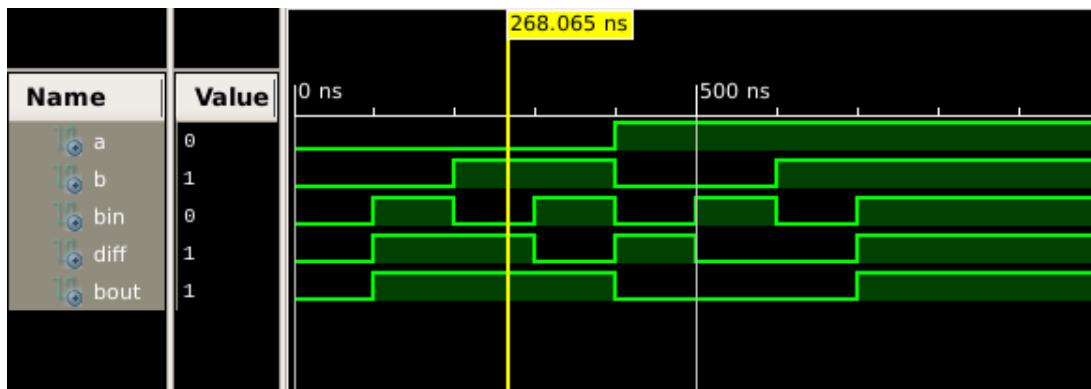


Figure 19: Simulation waveform of the Full Subtractor.

12. 8-bit Subtractor Design

An 8-bit Subtractor calculates $A - B$ for two 8-bit values. It uses eight 1-bit Full Subtractors connected in a ripple-borrow chain. The borrow output from one stage goes to the borrow input of the next stage.

- $A[7:0]$ – minuend
- $B[7:0]$ – subtrahend
- Bin – starting borrow

- Diff[7:0] – 8-bit difference
- Bout – final borrow, which becomes 1 when $A < B$

For bit i (0 to 7): $\text{Diff}_i = A_i \oplus B_i \oplus \text{Bin}_i$

$$\text{Bin}_{i+1} = (\neg A_i \cdot B_i) + (\neg(A_i \oplus B_i) \cdot \text{Bin}_i)$$

When A is smaller than B , the final borrow is 1 and the 8-bit result is shown in two's-complement form.

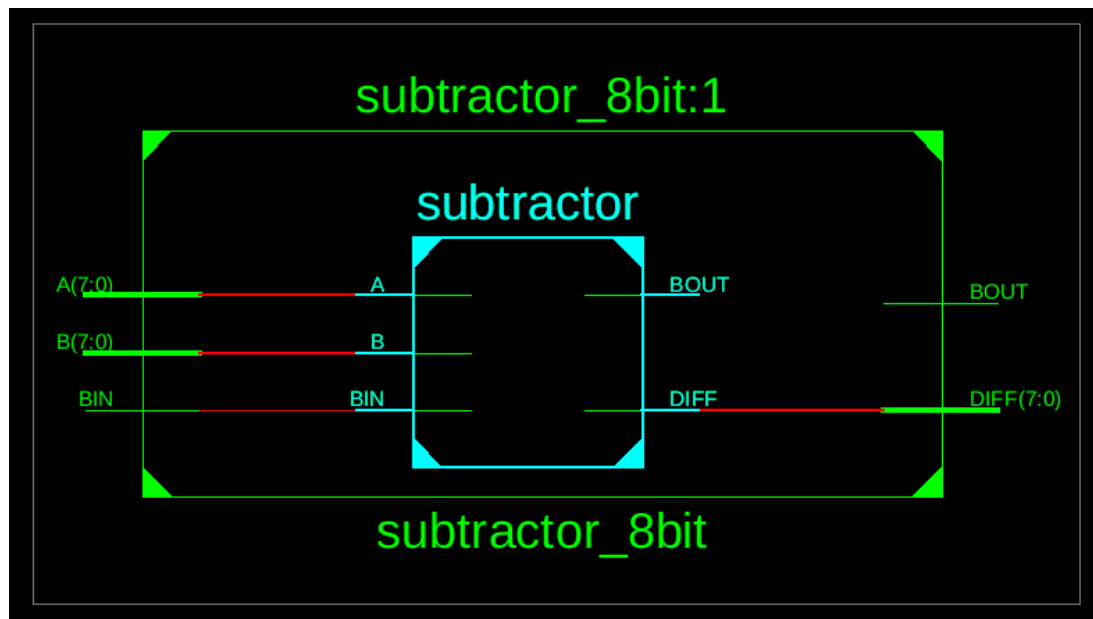


Figure 20: RTL schematic of the 8-bit Subtractor.

12. 8-bit Multiplexer (MUX) Design

A multiplexer selects one input from multiple inputs and sends the selected value to its output. In this ALU, an 8-bit 2-to-1 MUX chooses between the 8-bit Full Adder output and the 8-bit Subtractor output.

For a 1-bit 2-to-1 MUX: $Y = (\neg S \cdot I_0) + (S \cdot I_1)$

The same selection operation is applied to all eight bits using one common select signal S .

S	I0	I1	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

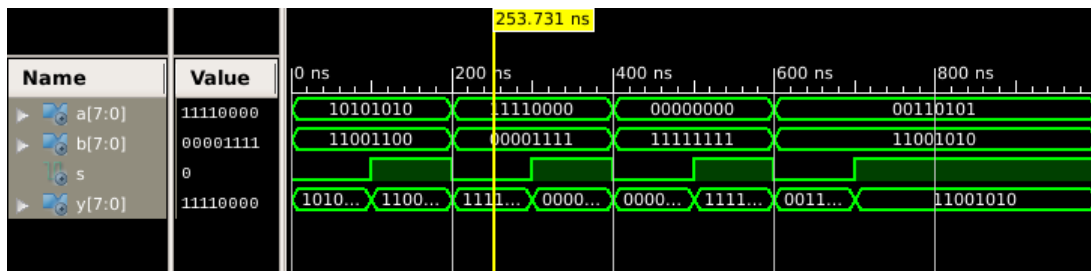


Figure 21: Simulation waveform of the 8-bit MUX.

13. ALU Design

The final 8-bit ALU combines the 8-bit Full Adder, the 8-bit Subtractor, and the 8-bit MUX. The same A and B operands are sent to both arithmetic blocks. The MUX then chooses the required result according to the select input.

- A[7:0] – first 8-bit operand
- B[7:0] – second 8-bit operand
- Sel – operation select
- Result[7:0] – selected 8-bit result
- Cout / Bout – carry for addition or borrow for subtraction

Sel	Operation	MUX selects	Result
0	Addition	8-bit Full Adder	A + B
1	Subtraction	8-bit Subtractor	A – B

Component hierarchy and NAND usage

Component	Built from	Number of NAND gates
NOT	NAND	1
AND	NAND	2
OR	NAND	3
XOR	NAND	4
Half Adder	1 XOR + 1 AND	6
Full Adder	2 Half Adders + 1 OR	15
8-bit Full Adder	8 Full Adders	120
Full Subtractor	2 XOR + 2 AND + 2 NOT + 1 OR	17
8-bit Subtractor	8 Full Subtractors	136
8-bit MUX	NAND-based gate structure	—
8-bit ALU	8-bit Full Adder + 8-bit Subtractor + 8-bit MUX	—

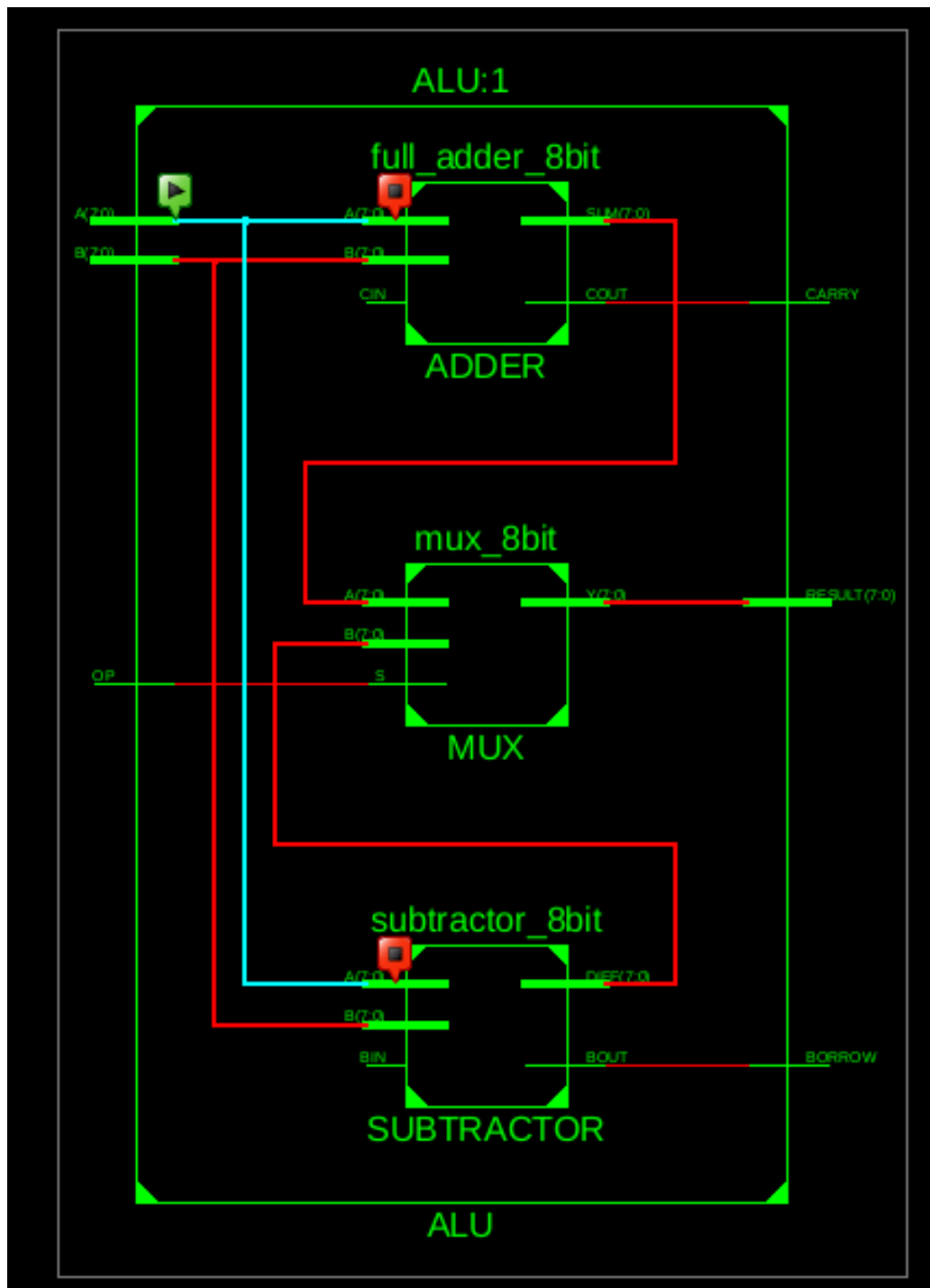


Figure 22: RTL schematic of the complete 8-bit ALU.

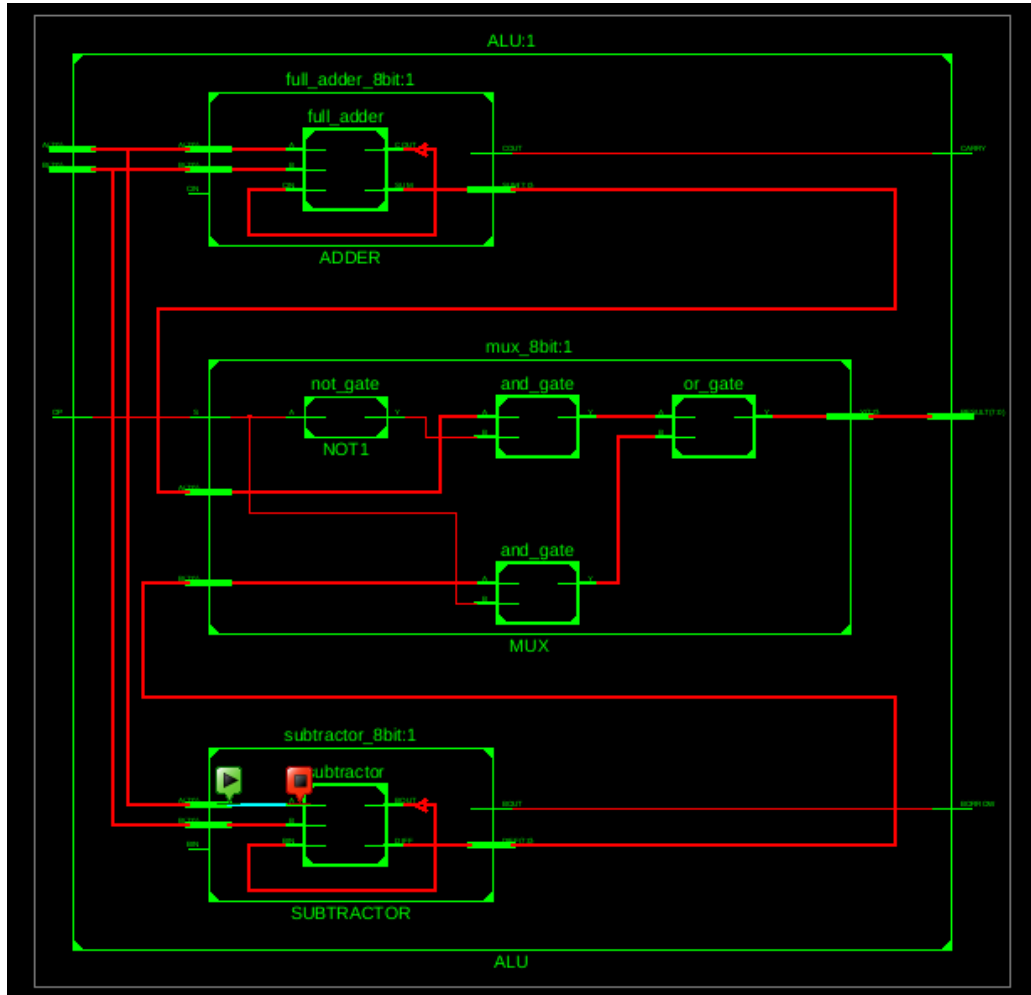


Figure 23: Expanded RTL schematic showing the internal ALU blocks.

14. RTL Schematic and Simulation

The complete ALU was tested in the ISim simulator included with Xilinx ISE. The testbench applies different 8-bit input pairs for both operations. Sel = 0 selects addition, while Sel = 1 selects subtraction. The Result and Cout/Bout outputs are then observed.

ALU Testbench Cases

No.	Sel	Operation	A	B	Result	Cout/Bout
1	0	$25 + 10 = 35$	00011001	00001010	00100011	0
2	0	$100 + 50 = 150$	01100100	00110010	10010110	0
3	0	$200 + 100 = 300$	11001000	01100100	00101100	1
4	0	$255 + 1 = 256$	11111111	00000001	00000000	1
5	1	$50 - 20 = 30$	00110010	00010100	00011110	0
6	1	$100 - 100 = 0$	01100100	01100100	00000000	0

7	1	$20 - 50 = -30$	00010100	00110010	11100010	1
8	1	$0 - 1 = -1$	00000000	00000001	11111111	1

Each test case is kept active for a fixed time so the corresponding output can be checked clearly.

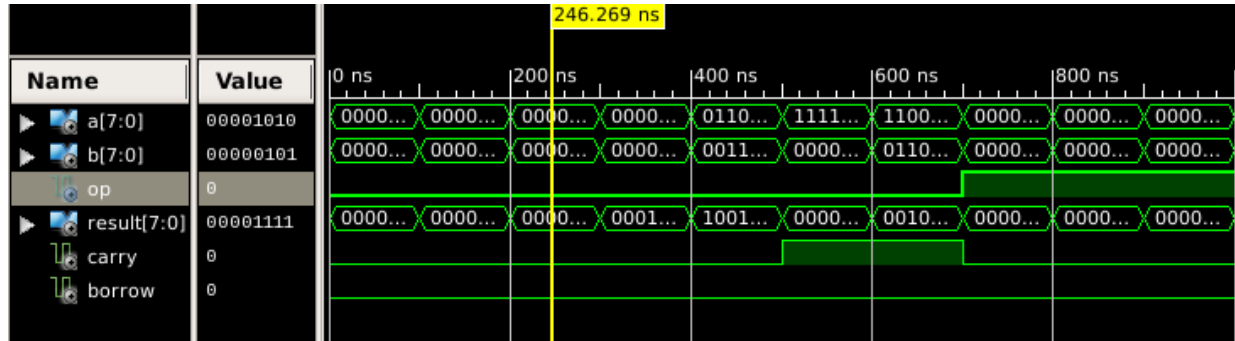


Figure 24: ISim waveform for ALU addition (Sel = 0).

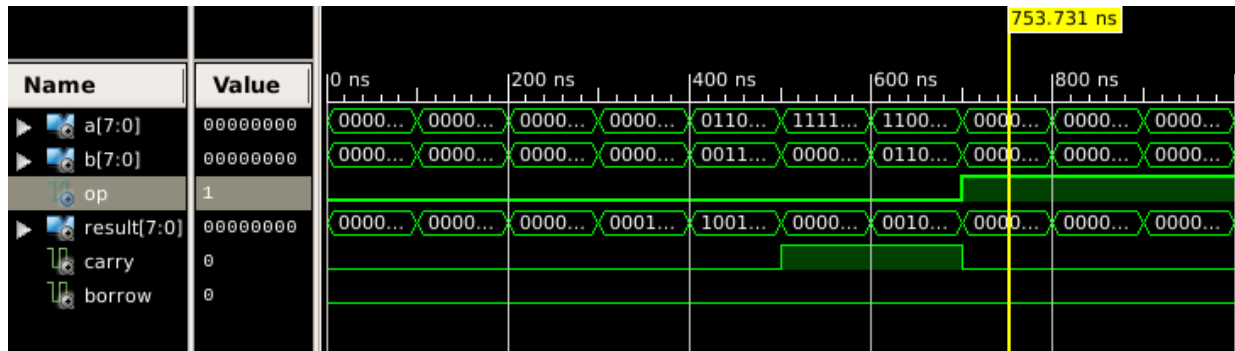


Figure 25: ISim waveform for ALU subtraction (Sel = 1).

For the waveform view, the important signals are A, B, Sel, Result, and Cout/Bout. Showing the buses in binary or decimal makes the values easier to verify.

15. Simulation Result Analysis

The simulation confirms that the ALU produces the expected outputs for both addition and subtraction.

Case 1: Normal addition (Sel = 0)

A = 25 (00011001), B = 10 (00001010). The result is 35 (00100011), and Cout = 0.

Case 2: Addition with carry (Sel = 0)

A = 200 (11001000) and B = 100 (01100100). The actual sum is 300, which is larger than the maximum 8-bit value. Therefore the stored result is 44 (00101100) and Cout = 1.

Case 3: Normal subtraction (Sel = 1)

A = 50 (00110010) and B = 20 (00010100). The result is 30 (00011110), and Bout = 0.

Case 4: Subtraction with borrow (Sel = 1)

$A = 20$ (00010100) and $B = 50$ (00110010). Since A is smaller than B , the result is negative. The 8-bit output is 226 (11100010), which represents -30 in two's-complement form. $Bout = 1$.

Overall, the simulation output matches the expected theoretical results for both addition and subtraction.

16. GitHub Repository

The VHDL source files, testbenches, and related project files are available in the GitHub repository:

<https://github.com/Rifah03/VLSI-ISE-14.7>

17. Conclusion

In this experiment, an 8-bit ALU capable of addition and subtraction was designed and simulated using VHDL and Xilinx ISE. A hierarchical structural design method was followed.

The work started with a NAND gate implemented using dataflow modeling. Because NAND is a universal gate, it was used to build NOT, AND, OR, and XOR gates. A Half Adder was then created from XOR and AND gates, and two Half Adders plus an OR gate were used to create a Full Adder. Eight Full Adders were connected to make the 8-bit Full Adder.

For subtraction, a Full Subtractor was created using two XOR gates, two AND gates, two NOT gates, and one OR gate. Eight Full Subtractors were connected to make the 8-bit Subtractor. An 8-bit MUX was then used to choose between the addition and subtraction results, completing the ALU.

The simulation results of the individual gates, adders, subtractors, MUX, and complete ALU matched the expected truth tables and calculated values. Therefore, the design was successfully verified.

This experiment also showed how structural VHDL, component instantiation, hierarchical design, and simulation can be used to build a larger digital circuit from smaller reusable blocks.